

NUCLEAR Re:Mind — Game Design Document

Survival City-Builder · Educational (Fusion Physics) · Isometric Engine: Unity 6 / C# · Length: 30 in-game days · Genre: Edu-Survival City-Builder Version: v4.0 (Developer Handoff Edition) · Project: NSC #28

อ่านก่อนเริ่ม (สำหรับโปรแกรมเมอร์): เอกสารนี้คือสเปกเกมฉบับเต็มสำหรับนำไปพัฒนาใน Unity 6 / C# โดยตรง — มีตัวเลขครบทุกระบบ, ตารางสมดุลเศรษฐกิจ (S7), สูตรเตาปฏิกรณ์ (S8) และโครงสร้างคลาส C# (S16) ค่าปรับสมดุลทั้งหมดรวมไว้ที่ **S18 (Tuning Constants)** ปรับทีเดียวจบ

สารบัญ (Table of Contents)

S	หัวข้อ	S	หัวข้อ
1	ภาพรวมเกม (Overview)	11	ประกาศฉุกเฉิน (Decrees)
2	Core Loop	12	ควิซ 10 ข้อ (Quizzes)
3	ระบบเวลา (Time System)	13	ไอเทม (Items)
4	ทรัพยากร (Resources)	14	เฟส & ฉากจบ (Endings)
5	ประชากร (Population)	15	★ หยุดเวลาตอนวางอาคาร
6	อาคาร (Buildings)	16	สถาปัตยกรรม C# (Architecture)
7	สมดุลเศรษฐกิจ (Balance)	17	UI / UX Spec
8	CORE TOWER (Reactor)	18	★ ค่าปรับสมดุล (Tuning)
9	Hope & Knowledge	19	คำถามที่ค้าง (Open Decisions)
10	วิกฤตทั้งหมด (Crises)		

1. ภาพรวมเกม (Overview)

NUCLEAR Re:Mind คือเกมสร้างเมืองแนวเอาตัวรอด (survival city-builder) มุมมอง isometric ที่สอน "ฟิสิกส์นิวเคลียร์/ฟิวชัน" ผ่านการเล่นจริง ผู้เล่นบริหารเมืองได้พายุรังสีให้รอด พร้อมเร่งสร้างเตาปฏิกรณ์ฟิวชัน (CORE TOWER) ให้ถึงค่า $Q \geq 1.0$ (จุดคุ้มทุน Breakeven) ภายใน 30 วัน เพื่อกางโล่พลาสมาป้องกันพายุรังสีระลอกสุดท้าย

4 เสาหลักการออกแบบ (Design Pillars)

- เรียนผ่านการเล่น (Learn by Playing) — เนื้อหานิวเคลียร์ฝังในวิกฤตจริง ผู้เล่นเข้าใจเพราะต้องใช้ ไม่ใช่ท่องจำ
- ฟิสิกส์จริง (Grounded Physics) — ทุกวิกฤตอิงหลักจริง: โทคาแมก/สนามแม่เหล็กคู่, ALARA, เวชศาสตร์นิวเคลียร์, การฉายรังสีอาหาร, ฟิวชัน D-T
- จังหวะกดดันแต่ให้คิด (Tense but Thoughtful) — มีนาฬิกาเดินจริงตอนผลิต แต่ช่วงวางแผน/ตอบควิซ/วางอาคาร "เวลาหยุด" ให้คิด
- การตัดสินใจมีต้นทุนจริง (Meaningful Choices) — ทุกทางเลือก A/B/C แลกมาด้วยทรัพยากร/คน/เวลา ไม่มีตัวเลือกฟรี

เงื่อนไขชนะ/แพ้

ผลลัพธ์	เงื่อนไข
ชนะ (True Ending)	ดัน CORE TOWER ถึง $Q \geq 1.0$ ภายในจบ Day 30 และมีคนรอด
ชนะบางส่วน (Normal Ending)	Day 30 จบที่ $Q\ 0.5\text{--}0.99$
แพ้ (Game Over)	Hope ต่ำ 0 ก่อน Day 30 · หรือเตา Meltdown ($HEAT \geq 100$) · หรือ Day 30 จบที่ $Q < 0.5$

2. Core Loop

ทุกวันผู้เล่นวน 5 ขั้นตอนเดิมซ้ำ ๆ การตัดสินใจเล็ก ๆ สะสมกลายเป็นชะตาของเมืองเมื่อถึง Day 30

[1] เริ่มวัน	→ รับสรุปสถานะ (คลัง, Hope, CORE%, HEAT, ประชากร)
[2] วางคนงาน	→ จัดสรรแรงงานลงโรงงาน/ภารกิจ
[3] ผลิตทรัพยากร	→ โรงงานทำงาน (Live Phase 60s)
[4] อัปเกรด/วิจัย	→ ลงทุนอนาคต (วางอาคาร = เวลาหยุด, ดู §15)
[5] รับเหตุการณ์ประจำวัน	→ วิกฤต/เหตุการณ์แต่ง → เลือก A/B/C → Quiz ยืนยันความเข้าใจ → วนใหม่

จุดสำคัญ — Quiz ผิงในลูป: เมื่อเกิดวิกฤตในขั้น [5] และผู้เล่นเลือกวิธีแก้ (A/B/C) เสร็จ ระบบจะแทรก **Decision Quiz** ยืนยันความเข้าใจต่อท้าย *ทันที* ก่อนสรุปวัน — คิวจึงไม่ใช่ระบบแยก แต่เป็น "ขั้นสุดท้ายของเหตุการณ์นั้น"

เกิดวิกฤต → เลือก A/B/C → ลงมือแก้ → + Quiz → คำนวณผล + Knowledge → สรุปวัน

3. ระบบเวลา & โครงสร้างวัน (Time System)

โครงสร้าง 1 วัน

แต่ละวันมี 2 เฟส:

เฟส	ระยะ เวลา	เกิดอะไร
Planning Phase	30 วินาที	ผู้เล่นจัดคนงาน, สั่งอัปเกรด, วางอาคาร, เลือกโหมดเตา (ล็อกคั้งทั้งวัน), อ่านข้อมูล
Live Phase	60 วินาที	เวลาเดิน, เตาดำเนินตามโหมดที่ล็อกไว้, เหตุการณ์/วิกฤตแต่งได้
จบวัน (End of Day)	ทันที	คำนวณผลผลิตทั้งวันที่เดียว (batch) → ทักการบริโภค → tick เตา (CORE%/HEAT) → เช็กวิกฤต +คิว → ขึ้นวันใหม่

1 วัน = 90 วินาที (Planning 30s + Live 60s)

การผลิต (Production) คำนวณแบบ "จบวันที่เดียว" — ไม่ใช่สะสมต่อเนื่องระหว่างวัน ตอนจบ Live Phase ระบบจะรวมผลผลิต/ค่าเดินระบบ/บริโภคของทั้งวันมาคำนวณครั้งเดียว ทำให้โค้ดเรียบและตัวเลขตรงกับตาราง §6–§7 พอดี

โหมดเตาเลือกตอน Planning แล้วล็อกค้างเดินตลอด Live — เปลี่ยนกลางวันไม่ได้ ต้องรอ Planning ของวันถัดไป (เป็นการตัดสินใจรายวันที่มีน้ำหนัก)

กฎเวลาสำคัญ (สำหรับ TimeManager)

นาฬิกา (ทั้ง Planning 30s และ Live 60s) เดินเฉพาะตอน **ไม่มีเหตุให้หยุด (pause reason)** ใด ๆ เลย — เหตุหยุดมีหลายแหล่งและต้องซ้อนกันได้ (ดูรายละเอียด §15, §16):

- กำลังวางอาคาร (Placement) → หยุด
- มี popup ควิซเปิดอยู่ → หยุด
- มี popup วิกฤตเปิดอยู่ → หยุด
- เปิดเมนูหลัก/หยุดเอง → หยุด

```
TimeManager.IsRunning == (activePauseReasons.Count == 0)
```

นาฬิกาวันเดินต่อเมื่อ `IsRunning == true` เท่านั้น (เปิด Build Menu หรือ popup ใด ๆ = เวลาหยุดทันที ไม่ว่าจะอยู่เฟสไหน)

ภาพรวมจังหวะ 30 วัน

- **Day 1** — Tutorial Day: ไม่มีวิกฤต สอน 3 อย่าง (คุม Shelter, เดินโรงงานพื้นฐาน 3 โรง, ฝึกคน)
- **Day 2–30** — วนลูปเติม วิกฤตและควิซดังตามตาราง (ดู §14 Victory Loop)
- จบ 1 รอบเกม ≈ 30–45 นาที

4. ระบบทรัพยากร 5 ชนิด + ค่าเริ่มต้น (Resources)

ค่าเริ่มต้นเกม (Day 1 Start)

ทรัพยากร	ค่าเริ่มต้น	เพดานคลัง (cap)
พลังงาน (Energy)	200	9999
น้ำ (Water)	150	9999
อาหาร (Food)	150	500*
แร่เหล็ก (Iron Ore)	100	9999
Deuterium	0	9999
Tritium	0	9999
Hope	100	100 (min 0)
Knowledge	0	100

* อาหารเก็บเกิน 500 จะ trigger วิกฤตเน่าเสีย (ถ้ายังไม่มี Agri Dome) — ดู §10 Crisis 3

บทบาททรัพยากร & อัตราการบริโภค

ทรัพยากร	บทบาท	บริโภคต่อวัน
พลังงาน	"เงิน" ของเกม + ค่าเดินระบบอาคาร · ใช้สร้าง/วิจัย/เปิดโซน/แก้วิกฤต · หมด = ทำอะไรไม่ได้	ตามค่าเดินระบบอาคาร (ดู S6)
น้ำ	คนดื่ม · หล่อเย็นเตา CORE · สกัด Deuterium · ขาด = คนป่วย + เตาร้อนเกิน	2 / คน / วัน + ค่าหล่อเย็นเตา
อาหาร	คนกิน · ใช้ฝึกคนชั้นสูง · ขาด = Hope ดิ่ง เสี่ยงจลาจล	2 / คน / วัน
แร่เหล็ก	วัตถุดิบสร้าง/ซ่อม/คราฟต์ไอเทม (Rad-Gear, ชิ้นส่วน CORE) · ขุดลึก = เสี่ยงรังสี	— (ใช้ตามต้องการ)
เชื้อเพลิงฟิวชัน	ดันค่า Q ของเตา	ตามโหมดเตา (ดู S8)

น้ำหล่อเย็นเตาใช้คลั่งเดียวกับน้ำดื่ม — ไม่มีถึงน้ำแยก ดังนั้นยิ่ง Boost เตา (กินน้ำหล่อเย็นมาก) ยิ่งแย่งน้ำดื่มของประชากร ต้องบาลานซ์โรงน้ำให้พอทั้งสองทาง

เชื้อเพลิงฟิวชัน 2 ชนิด:

เชื้อเพลิง	ระดับ	แหล่งผลิต	ใช้เมื่อ
Deuterium (2H)	พื้นฐาน	สกัดจากน้ำ (โรงน้ำ L3)	ป้อนเตาเริ่ม Day 11 (เฟส 2–3 ของเตา)
Tritium (3H)	ชั้นสูง หายาก	Zone B (Phase 4)	ดัน Q เยอะสุด ช่วงพายุท้ายเกม

5. ระบบประชากร & คลาส (Population)

ผู้เล่นจัดคนที่มีอยู่ลงทำงานแต่ละระบบ เริ่มเกมมี 10 คน (ทั้งหมดเป็น Worker) บ้าน (Shelter) ขยายเพิ่มเพดานประชากรทีหลัง

3 คลาส (Classes)

คลาส	หน้าที่	ได้มาจาก	ต้นทุนฝึก
Worker (คนงาน)	ผลิตทุกอย่าง (โรงไฟ/น้ำ/อาหาร) + ขุดเหมือง — กำลังหลัก	เริ่มมี 10 + เพิ่มตาม Shelter	ฟรี (เกิดเมื่อขยาย Shelter)
Engineer (วิศวกร)	วิจัย + เปิดโซน + ค่อม CORE TOWER + หล่อเย็น — ขาดวิศวกร เตาเดินไม่ได้	ฝึกจาก Worker ที่ห้องวิจัย	Food 30 + Energy 50 / คน · ใช้เวลา 1 วัน
Medic (แพทย์)	รักษา + กันรังสีในวิกฤตโรค · ลดผลกระทบรังสี · ช่วยกัน Hope ดิ่ง	ฝึกจาก Worker ที่โรงพยาบาล	Food 40 + Energy 60 / คน · ใช้เวลา 1 วัน

ฝึกคลาส = แปลง Worker 1 คน → คลาสนั้น (จำนวนประชากรรวมเท่าเดิม) ต้องมีอาคารปลดล็อกก่อน (Research Lab → Engineer, Hospital → Medic)

Shelter (เพดานประชากร)

ระดับ	เพดานประชากร	ต้นทุนสร้าง/อัป	ปลดล็อก
L1	10	(เริ่มมี)	Phase 1
L2	20	Iron 60 + Energy 100	Phase 1
L3	40	Iron 120 + Energy 250	Phase 2
L4	80	Iron 200 + Energy 450	Phase 3

กลไกเติมประชากร: เมื่อมีอาหารเหลือพอและ Hope ≥ 50 ประชากรจะเพิ่มอัตโนมัติ **+1 คน / วัน** จนเต็มเพดาน Shelter (ถ้าอาหารขาดหรือ Hope < 50 หยุดเติม) — สร้างแรงกดดันให้ต้องทำฟาร์มก่อนจะโตได้

6. อาคารทั้งหมด + ตัวเลขครบ (Buildings)

นิยามคอลัมน์: - **ผลิต/วัน** = output ต่อวัน (ตอนมีคนงานครบ) - **คน** = จำนวนแรงงานที่ต้องวางประจำ (engineer สำหรับ Research/CORE) - **ต้นทุนสร้าง/อัป** = จ่ายครั้งเดียวตอนสร้างหรืออัปเกรด - **ค่าเดินระบบ/วัน** = ทักทุกวันตอนเดินเครื่อง (upkeep)

อาคารผลิต (Production)

อาคาร	ระดับ	ผลิต/วัน	คน	ต้นทุนสร้าง/อัป	ค่าเดินระบบ/วัน
โรงไฟ (Power Plant)	L1	+60 Energy	1	เริ่มมี 1 หลัง / หลังถัดไป Iron 40	—
	L2	+180 Energy	2	Energy 150 + Iron 40	—
	L3	+450 Energy	3	Energy 360 + Iron 80	—
โรงน้ำ (Water Plant)	L1	+50 Water	1	Iron 30	Energy 10
	L2	+150 Water	2	Energy 150 + Iron 40	Energy 25
	L3	+375 Water	3	Energy 360 + Iron 80	Energy 50 · ปลดสกัด Deuterium
โรงอาหาร (Food Plant)	L1	+40 Food	1	Iron 30	Energy 8 + Water 15
	L2	+120 Food	2	Energy 150 + Iron 40	Energy 16 + Water 30
	L3	+300 Food	3	Energy 360 + Iron 80	Energy 30 + Water 60
เหมือง (Mine)	L1	+30 Iron	2	Iron 20	Energy 12
	L2	+90 Iron	2	Energy 120 + Iron 40	Energy 20 · เปิด Zone A
	L3	+225 Iron	3	Energy 360 + Iron 80	Energy 36 · เสี่ยงรังสี↑ · trigger Mutation ได้
Agri Dome	—	+150 Food	3	Iron 150 + Energy 300	Energy 40 + Water 40 · กันอาหารเน่า

อาคารวิจัย/ระบบ/สนับสนุน

อาคาร	ผล	คน	ต้นทุนสร้าง	ค่าเดินระบบ/วัน	ปลดล็อก
Shelter	เพิ่มเพดานประชากร (ดู S5)	—	(ตาม S5)	—	Phase 1
ห้องวิจัย (Research Lab)	ปลดล็อกเทค · ฟีก Engineer · สกัด Deuterium · วิจัยยา/เมล็ดพันธุ์	2 (eng)	Iron 80 + Energy 120	Energy 20	Phase 2
โรงพยาบาล (Hospital)	รักษาคนป่วย · กันรังสี · ฟีก Medic	2	Iron 100 + Energy 150	Energy 15	Phase 3
Zone A (เหมือง ลึก)	ขุดแร่หายาก (เสี่ยงรังสี)	—	ปลดผ่านอัป Mine L2 + Energy 100	—	Phase 3
Zone B (บ่อ เชื้อเพลิง)	ผลิต Tritium	3 (eng)	Iron 150 + Energy 200	Energy 30 + Water 30	Phase 4
★ CORE TOWER	เตาปฏิกรณ์ — ต้นค่า Q (ดู S8)	2–4 (eng)	Iron 200 + Energy 200 + วิจัย	ดู S8 (เชื้อเพลิง+ น้ำหล่อเย็น)	Phase 3

ระบบหล่อเย็น CORE (Magnetic Confinement) — ส่วนต่อขยายของ CORE TOWER

อุปกรณ์	ผล (อิงสูตร S8)	คน	ต้นทุน/ระดับ	คอนเซ็ปต์ฟิสิกส์
Toroidal Coils	+1 ระดับ <code>coolingTowerLevel</code> (มีผล x10 ในสูตรหล่อเย็น, cap 3 ระดับ)	1 (eng)	Iron 50 + Energy 80	สนามแม่เหล็กรอบ บีบพลาสมาให้ รឹងเป็นวงในเตา = ยกเพดานหล่อเย็น
Poloidal Coils	เปิดเทอม <code>coolingEngineers</code> × 4 + กัน micro-damage ตอน HEAT 80–99	ใช้ eng ประจำ	Iron 60 + Energy 100	กันพลาสมาไม่ให้ชนผนัง = เสถียร ลดความร้อนรั่ว

หัวใจการเล่น: น้ำ + วิศวกร + คอยล์ คือสิ่งที่ทำให้ "Boost เตาได้โดยไม่ Meltdown" ยิ่งลงทุนหล่อเย็น ยิ่งดัน Q ได้เร็วและปลอดภัย

7. ตารางพิสูจน์สมดุลเศรษฐกิจ (Economy Balance)

ส่วนนี้พิสูจน์ว่า "ถ้าเล่นถูกทาง คลังไม่ติดลบ" และแสดงว่าแต่ละเฟสกดดันแค่ไหน — ตัวเลขสุทธิต่อวัน (net/day) ของแต่ละเฟส (สมมติเดินเครื่องครบ คนงานพอ):

Phase 1 (Day 1–5) · 10 คน · อาคาร L1 พื้นฐานครบ 4 โรง

อาคาร: โรงไฟ L1, โรงน้ำ L1, โรงอาหาร L1, เหมือง L1 — ใช้คน 1+1+1+2 = 5 คน (เหลือ 5 คนไว้สร้าง/ฟีก)

ทรัพยากร	ได้	เสีย	
Energy	+60	-10 (น้ำ) -8 (อาหาร) -12 (เหมือง)	+30 ✓
Water	+50	-15 (อาหารใช้) -20 (คนดื่ม 10×2)	+15 ✓
Food	+40	-20 (คนกิน 10×2)	+20 ✓
Iron	+30	—	+30 ✓

→ ทุกอย่างเป็นบวก เมืองยั่งยืน ผู้เล่นเอา Iron/Energy ส่วนเกินไปสร้างห้องวิจัย/ขยาย Shelter

Phase 2 (Day 6–10) · ~10 คน (2 เป็น Engineer) · สร้างห้องวิจัย, อัปโรงไฟ/น้ำ L2

อาคาร: โรงไฟ L2, โรงน้ำ L2, โรงอาหาร L1, เหมือง L1, ห้องวิจัย

ทรัพยากร	ได้	เสีย	
Energy	+180	-25 (น้ำ L2) -8 (อาหาร) -12 (เหมือง) -20 (วิจัย)	+115 ✓
Water	+150	-15 (อาหาร) -20 (ดื่ม)	+115 ✓ (เก็บไว้หล่อเย็น/สกัด Deuterium)
Food	+40	-20 (กิน) -ฝึก eng เป็นครั้ง	+20 ✓
Iron	+30	- สร้าง/อัป	บวก

→ สะสมส่วนเกินไว้ลงทุน Phase 3

Phase 3 (Day 11–20) · 20 คน · CORE TOWER ออนไลน์ + Hospital + Zone A · วิกฤตเริ่มตึง

อาคาร: โรงไฟ L3, โรงน้ำ L3, โรงอาหาร L2, เหมือง L2, ห้องวิจัย, โรงพยาบาล, CORE TOWER + คอยล์ บริโภค 20 คน = Food 40 + Water 40 / วัน

ทรัพยากร	ได้	เสีย (รวมเตา+หล่อเย็น)	
Energy	+450	-50(น้ำ) -16(อาหาร) -20(เหมือง) -20(วิจัย) -15(รพ.) ≈ -40(เตา/คอยล์)	≈ +289 ✓
Water	+375	-30(อาหาร) -40(ดื่ม) -50(หล่อเย็นเตา) -30(สกัด Deuterium)	≈ +225 ✓
Food	+120	-40(กิน)	+80 ✓
Iron	+90	- คราฟต์/สร้าง	บวก

→ งบบวกหนา **ตั้งใจ** ให้รับแรงกระแทกจากวิกฤตได้ (เช่น Crisis A หัก Energy -300 ทันที, Crisis C หักน้ำ -50%) ส่วนเกินคือกันชน

Phase 4 (Day 21–30) · สูงสุด 40 คน · Zone B + Tritium + พายุ · ช่วงตึงที่สุด (climax)

บริโภค 40 คน = Food 80 + Water 80 / วัน · ต้องอัปโรงอาหาร/น้ำ L3 (หรือ Agri Dome) ความต้องการพลังงานพุ่งสุด (เตา Boost + Zone B + หล่อเย็นหนัก) — อาจต้องโรงไฟ L3 สองหลัง (+900 Energy)

ทรัพยากร	สถานะ
Energy	ตึง — ต้องบริหาร/สะสมล่วงหน้า เป็นแรงกดดันหลักของพายุ
Water	ตึง — หล่อเย็นหนัก + Tritium + ดื่ม
Food/Iron	จัดการได้ถ้าอัปครบ

→ **เฟสนี้ตั้งใจให้ตึง** เป็นโคลแม็กซ์ ระบบประกาศฉุกเฉิน (S11) ให้แรงงานหล่อเย็นเสริมแลกกับ Hope

คำแนะนำสำหรับ dev: ตัวเลขข้างบน "สมดุลบนกระดาษ" (consistent + ยืนยันแบบมีแรงกด) ค่าจริงควร **playtest แล้วจนที่ §18** โดยเฉพาะ: ระยะกันชนของ Phase 3 (ลด surplus ลงง่ายขึ้น) และความตึงพลังงาน Phase 4 (เพิ่ม/ลด output โรงไฟ L3)

8. CORE TOWER — ระบบเตาปฏิกรณ์ + สูตรคณิต (Reactor)

หัวใจของเกม เช็ตอัฟในยุคเดียว — ค่าวัดคือ **CORE% (0–100%)** ดันถึง 100% = ค่า Q ถึง 1.0 = เตากำเนิดโล่พลาสมา = ชนะ เริ่มเดินเครื่อง Day 11

ภาพรวม 3 เฟสของเตา

	เฟส 1	เฟส 2	เฟส 3 ★
ชื่อ	Cold Assembly	Plasma Ramp	Ignition
CORE%	30→50%	50→80%	80→100%
ค่า Q	≈0 → 0.10–0.30	0.30–0.60	0.60 → 1.0
วัน	Day 11–17	Day 18–24	Day 25–30
เชื้อเพลิง	แร่เหล็ก + วิศวกร (ประกอบ)	Deuterium	Tritium

เตาเริ่มที่ **CORE% = 30%** ตอนสร้างเสร็จ Day 11 (ผ่านการประกอบด้วยแร่เหล็ก+วิศวกร) แล้วผู้เล่นดันต่อด้วยเชื้อเพลิง

โหมดเร่งเครื่อง (Overclock Modes)

ทุกวันผู้เล่นเลือก 1 ใน 4 โหมด ยิ่งเดินแรง CORE% โตเร็วแต่ HEAT พุง:

โหมด	ตัวคูณ (modeMultiplier)	ความร้อนจากโหมด (heatFromMode)
Idle (พัก)	x0	+0
Normal (ปกติ)	x1	+5
Boost (เร่ง)	x2	+20
Overdrive (เร่งสุด)	x3	+40 (+ เสี่ยง micro-damage)

★ สูตรคณิต (Reactor Math)

1) การเพิ่ม CORE% ต่อวัน:

$$\Delta \text{CORE\%} = 3 \times \text{modeMultiplier} \times \text{fuelEfficiency}$$
$$\text{fuelEfficiency} = \min(1, \text{fuelInput} \div \text{fuelNeed}) + \text{knowBonus}$$
$$\text{knowBonus} = (\text{Knowledge} \geq 80) ? 0.10 : 0$$

- fuelNeed = เชื้อเพลิงที่โหมดต้องการต่อวัน (เช่น Normal = 10, Boost = 20, Overdrive = 30 หน่วย — ดู §18)
- ถ้าเชื้อเพลิงไม่พอ fuelEfficiency < 1 → CORE% โตช้าลงตามสัดส่วน

2) การเปลี่ยนแปลง HEAT ต่อวัน:

$$\Delta \text{HEAT} = \text{heatFromMode} + \text{heatFromStorm} - \text{cooling}$$

$$\begin{aligned} \text{cooling} = & 15 \\ & + (\text{waterUsedForCooling} \div 10) \\ & + (\text{coolingEngineers} \times 4) \\ & + (\min(\text{coolingTowerLevel}, 3) \times 10) \end{aligned}$$

$$\begin{aligned} \text{heatFromStorm} = & 0 && (\text{ก่อน Day 25}) \\ = & +12 / \text{วัน} && (\text{Day 25-30, ช่วง Ignition/พายุ}) \end{aligned}$$

- `coolingTowerLevel` = จำนวนระดับ Toroidal Coils (cap มีผล 3 = +30 หล่อเย็น)
- `coolingEngineers` = วิศวกรที่วางประจำหล่อเย็น (ต้องมี Poloidal Coils เปิดถึงจะนับเทอมนี้)
- HEAT มีพื้น 0 (floored) และเพดานเหตุการณ์ที่ 100

กลไก HEAT (โซนความร้อน)

HEAT	สถานะ
0-79	ปกติ ปลอดภัย
80-99	เตือนไฟ — เสี่ยง micro-damage (CORE% -2/วัน ถ้าไม่มี Poloidal Coils ป้องกัน)
≥100	MELTDOWN = แพ้ทั้งเกม

SCRAM (เบรกฉุกเฉินดับเตา)

- กดได้เมื่อ HEAT ≥ 90 (หรือกดป้องกันล่วงหน้าได้)
- ผล: **HEAT -40 · CORE% -10 · เสียน้ำ 50 · cooldown 2 วัน**
- เป็นเบรกฉุกเฉิน ไม่ใช่กลยุทธ์หลัก (เสีย CORE% ที่อุตสาหกรรม)

ดาบ 2 คม (Double-Edged) — แขนกลางความตึง

- **FAIL · TOO SLOW** — อยู่ Normal/Idle ตลอด HEAT ต่ำ แต่ % ไม่ถึง 100 ก่อนพายุจบ → แพ้
- ★ **SAFE CORRIDOR** — สลับ Boost ดัน % ในกรอบที่หล่อเย็นรับไหว ไม่แตะ Meltdown → ชนะ
- **FAIL · MELTDOWN** — Overdrive รัว % โตเร็วแต่ HEAT พุ่งเกิน 100 → เตาดับ → แพ้

Q Factor คืออะไร (ฟิสิกส์จริง)

Q = อัตราส่วน พลังงานที่ฟิวชันปล่อยออก ÷ พลังงานที่ป้อนเข้า · **Q = 1** คือ "**จุดคุ้มทุน (Breakeven)**" จุดเปลี่ยนสำคัญของงานวิจัยฟิวชันทั้งโลก เกมแมป CORE% 100% ↔ Q 1.0 เพื่อให้ผู้เล่นเข้าใจเป้าหมายนี้

★ Worked Example — ทำไมต้อง Boost (พิสูจน์สมมูลเตา)

เตาเริ่ม 30% Day 11 ต้องถึง 100% ภายใน Day 30 = ดัน 70 จุด ใน 20 วัน (Day 11-30)

- **เดิน Normal ล้วน (เชื่อเพลิงเต็ม):** $\Delta \text{CORE\%} = 3 \times 1 \times 1 = 3/\text{วัน} \times 20 \text{ วัน} = 60 \text{ จุด} \rightarrow$ จบที่ **90%** ไม่ถึง
- **Normal + knowBonus (Knowledge≥80):** $3 \times 1 \times 1.1 = 3.3/\text{วัน} \times 20 = 66 \rightarrow$ จบที่ **96%** ยังขาดนิดเดียว
- **Normal เป็นหลัก + Boost ~4 วัน:** $\approx (16 \times 3.3) + (4 \times 6.6) = 52.8 + 26.4 = 79.2 \text{ จุด} \rightarrow$ เกิน 100% ✓ **ชนะ**

HEAT ของแผน Boost: - Boost = +20 heat/วัน ถ้าหล่อเย็นแค่พื้น 15 → net +5/วัน สะสมช้า ๆ ถึงโซนเตือน - ลงทุนหล่อเย็น (วิศวกร 2 + น้ำ 50 + Toroidal L3): cooling = 15 + 5 + 8 + 30 = **58/วัน** → Boost net = 20-58 = **-38** → HEAT แทบไม่ขึ้น **Boost ได้ปลอดภัย ✓ - ช่วงพายุ (Day 25-30):** +12 heat บังคับ ถ้าหล่อเย็นพื้น 15 + Boost = 20+12-15 = +17/วัน → Meltdown ใน ~6 วัน **△ → ต้องมีหล่อเย็น** (กับ Decree เสริมแรงงานหล่อเย็น §11) ถึงจะ Boost รอดพายุ

→ บทสรุปดีไซน์: **เดินเฉย ๆ ไม่ชนะ ต้อง Boost แต่ Boost ต้องลงทุนหล่อเย็นก่อน** = วงจรกลยุทธ์หลักของเกม

9. Hope & Knowledge

Hope (ขวัญเมือง)

เริ่ม 100 · max 100 · min 0 = Game Over

เหตุการณ์	ผล Hope
อาหารขาด (บริโภคไม่พอ)	-10 / วัน
คนงานเสียชีวิต	-5 / คน
แก้วิกฤตสำเร็จ	+5 / วิกฤต
มี Medic ประจำ + ไม่มีของขาด	+2 / วัน (พื้นตัว)
Decree 1 (เกณฑ์ผู้ป่วย)	-8 ทันที, -3 / วัน ระหว่างใช้
Decree 2 (แรงงานเด็ก)	-15 ทันที

Knowledge (ความเข้าใจนิเวศียร์)

ควิซตอบบังคับ — ห้ามไม่ได้ ผู้เล่นต้องเลือกคำตอบก่อนจึงปิด popup และเดินเกมต่อได้ (ไม่มีปุ่มข้าม) ตอบผิดยังได้แต้มบางส่วนเพราะได้อ่านเฉลย (fail forward) จึงไม่ลงโทษคนที่ยังไม่รู้ แต่บังคับให้ "อ่าน + ตัดสินใจ" ทุกครั้ง = Knowledge before Choice

การกระทำ	ได้ Knowledge
ตอบ Decision Quiz ถูก	+8 / ข้อ (10 ข้อ = สูงสุด ~80)
ตอบ Decision Quiz ผิด	+3 / ข้อ (ยังได้เพราะต้องอ่านเฉลย — fail forward)
อ่าน Codex / entry	+2 / รายการ
ผ่านวิกฤตสำเร็จ	+5 / วิกฤต

Knowledge Thresholds (โบนัส — ได้ทางเลือก ไม่ลงโทษ)

Knowledge	ระดับ	โบนัส
0-29	Novice	ไม่มีโบนัส (เริ่มได้ปกติ ไม่ตัดรอบ)
30-59	Aware	ค่าพลังงานในการวิจัย -10%
60-79	Skilled	วิศวกรทำงานเร็วขึ้น (ฝึก/วิจัยไวขึ้น)
80-100	Expert	CORE Efficiency: knowBonus = +0.10 ต่อ ΔCORE% (ดัน Q ไวขึ้นช่วง FIRST LIGHT)

★ คลังความรู้ถาวร (Knowledge Persistence / Meta-Progression)

Knowledge และ Codex ที่ปลดล็อกแล้วจะถูกบันทึกถาวร ข้ามรอบเล่น — ต่างจากทรัพยากรอื่นที่รีเซ็ตทุกครั้ง

- เมื่อผู้เล่น **แพแล้วเริ่มเกมใหม่** → ทรัพยากรทุกอย่างรีเซ็ตกลับค่าเริ่มต้น (Energy/Water/Food/Iron/เชื้อเพลิง/ Hope/อาคาร/ประชากร = S4, S5) ยกเว้น **"คลังความรู้" (Knowledge bank + Codex ที่อ่านแล้ว) ที่คงอยู่**
- ผลคือ ยิ่งเล่นซ้ำ ผู้เล่นยิ่งสะสมความเข้าใจ และได้โบนัส Threshold (เช่น ถ้าเคยถึง Knowledge ≥ 80 มาแล้ว รอบใหม่ จะได้ knowBonus +0.10 ตั้งแต่ต้น) → ช่วยให้ผู้เล่นที่ตั้งใจเรียน "เก่งขึ้นจริง" และผ่านได้ในรอบถัด ๆ ไป
- เก็บค่านี้ไว้นอกเซฟรอบปกติ (เช่น PlayerPrefs หรือไฟล์ meta แยก) — ดูคลาส MetaProgress ใน S16

10. วิกฤตทั้งหมด (Crises)

ทุกวิกฤตมีโครงสร้าง: เกิดวิกฤต → เลือกทางแก้ A/B/C (ไม่มีทางฟรี) → ลงมือ → ♦ Quiz (ตอบบังคับ) แต่ละทางเลือกแลกคนละอย่าง (พลังงาน/คน/เวลา)

วิกฤต 1 · เสถียรภาพพลาสมา (Plasma Instability)

เนื้อหาที่สอน: ในโทคาแมก พลาสมาหลายร้อยล้านเคลวินถูกกักด้วยสนามแม่เหล็กคู่ (Toroidal + Poloidal) ไม่ให้ชนผนังเตา ถ้าสนามจ่ายไฟไม่นิ่ง พลาสมาจะบิดเบี้ยวและถ่ายเทความร้อนเข้าตัวอาคาร เสี่ยง Meltdown

เงื่อนไขเกิด: HEAT > 80 หรือ $Q > 0.3$ (จบเฟส 1) → หน้าจอแดงกะพริบ ไซเรนดัง · มีเวลา 3 วันก่อนเตาระเบิด · ~Day 17

ทางเลือก	ต้นทุน	ผล
A · เร่งสนามแม่เหล็กวงแหวน	พลังงาน -300 หน่วย · ย้ายคน 3 คนเผ่าห้องคุมไฟ 1 วัน	สำเร็จทันที เตาปลอดภัย แต่ขาดพลังงานหลายวัน
B · ซ่อมขดลวดด้วยมือ	วิศวกร 4 คน ติด 2 วัน · แร่เหล็ก 150 · เสี่ยงป่วย 50%	ประหยัดไฟ แต่เสี่ยงเสียแรงงานถาวร + Hope ลด
C · ฉีดสารหล่อเย็นฉุกเฉิน	น้ำสะอาด -50% ของคลัง · ค่า Q -20%	ผ่านง่ายสุดตอนแรก แต่เกิดวิกฤตน้ำตามมา + ขยับใกล้เส้นตาย

→ ผูกควิซ: Q2, Q3

วิกฤต 2 · โรคร้ายระบาด (Malignant Outbreak)

สถานการณ์: ฝุ่นรังสีรั่วชั่วคราว ทำให้คนงานที่ชุดแรในโซน A เกิดเซลล์กลายพันธุ์ (เนื้อร้ายลุกลาม) ป่วยพร้อมกัน 15 คน → การผลิตอาหาร/น้ำหยุดชะงัก มีเวลา 3 วันก่อนคนงานเสียชีวิต

เงื่อนไขเกิด: ส่งคนชุดโซน A มากเกินไป → คนงาน 15 คนป่วยกะทันหัน · ~Day 20

ทางเลือก	ต้นทุน	ผล
A · สแกนคัดกรอง (PET/SPECT)	พลังงาน 200 · คนระดับสูง 2 คนคุมเครื่อง	คัดกรองแม่นยำ รักษาหายทันที 10 คน · อีก 5 คนต้องไปต่อทางเลือก B
B · บำบัดด้วยสารเภสัชรังสี	วัสดุแล็บ 200 · ใช้ฟลักซ์นิวตรอนจากเตา (เตาเดินโหมด Idle 1 วัน) ผลิตไอโซโทปการแพทย์	รักษากรบ 15 คนใน 1 วัน กลับมาทำงานทันที แต่วันนั้นเตา Idle (CORE% ไม่ขึ้น เสียจังหวะดัน Q)
C · ข่าเชื้อแกมมา + กักตัว	ไม่ใช้พลังงาน/Tritium · คน 15 คนกักตัว 4 วัน	เสี่ยงเสียชีวิต 3 คน · ขาดแรงงานทำฟาร์ม → อาหารหมด → ลูบฟังเป็นโดมิโน

→ ผูกควิช: Q4, Q5 (Q5 แต่งตอนส่งคนเข้าโซน A ครั้งแรก)

วิกฤต 3 · เสบียงเน่า (Food Crisis)

เนื้อหาที่สอน: (1) ปรับปรุงพันธุ์พืชด้วยรังสีแกมมา/นิวตรอน เช่น ข้าว กข6/กข10/กข15 ที่ทนทานให้ผลผลิตสูง · (2) ถนอมอาหารด้วยรังสีแกมมาจากโคบอลต์-60 ทำลายจุลินทรีย์/เชื้อรา ชะลอการเน่าเสีย

เงื่อนไขเกิด: อาหารเก็บ > 500 หรือไม่มี Agri Dome → อาหารเน่าเร็วขึ้น 3 เท่า · มีเวลา 3 วัน · ~Day 24

ทางเลือก	ต้นทุน	ผล
A · เพาะเมล็ดกลายพันธุ์	แร่เหล็ก 250 · นักวิจัย/วิศวกร 3 คนคุมแล็บ	ปลดล็อกแปลงพิเศษ ผลผลิตพุ่งใน 2 วัน — แก้ที่ต้นตอยั่งยืน
B · ฉายรังสีถนอมด้วยโคบอลต์-60	พลังงาน 300 · คนงาน 4 คนขนเสบียง	อัตราเน่าเหลือ 0% ทันที ล็อกอาหารไว้จนจบเกม แต่เสียพลังงานมาก
C · ลดปันส่วนอาหาร	ไม่ใช้พลังงาน/แร่เหล็ก	คนงานขาดสารอาหาร ประสิทธิภาพ -50% · Hope ดิ่งฮวบเสี่ยงจลาจลก่อนพายุ

→ ผูกควิช: Q6 (ถ้าเลือก A), Q7 (ถ้าเลือก B)

พายุ / FIRST LIGHT (Day 25–30) — ไม่ใช่ตัวเลือก แต่เป็นแรงกดดันถาวร

พายุรังสีมาถึง บังคับ +12 HEAT/วัน ตลอด Ignition (ดู S8) ผู้เล่นต้องดัน CORE% 80 → 100% ขณะสู้ความร้อน — เปิดตัว Tritium + Decree (S11) → ผูกควิช Q8, Q9, Q10

11. ประกาศฉุกเฉิน (Emergency Decrees)

ช่วงเฟส 3 (Day 25–30) พายุมา แรงงานหล่อเย็นขาด — ผู้เล่นเลือกประกาศใช้หรือไม่ก็ได้ แต่ละข้อให้ "แรงงานหล่อเย็นเพิ่มทันที" แลกกับ Hope ที่ดิ่ง

Decree	ได้	จ่าย	เสี่ยง
1 · เกณฑ์ผู้ป่วยร่วมงาน	ผู้ป่วยออกมาทำงาน → +หล่อเย็น ~2/คน	Hope -8 ทันที, -3/วัน	ผู้ป่วยตาย 20%/วัน
2 · ดึงแรงงานเด็ก	เด็กทำงานเบา/หล่อเย็น → +หล่อเย็น ~12 รวม	Hope -15 ทันที	—

หมายเหตุสรุป: ทั้ง 2 ข้อให้ "แรงงานหล่อเย็นเพิ่ม" แลกกับ Hope ที่ดิ่ง — เป็นการตัดสินใจที่มีต้นทุนจริงในเกม และ Q10 (S12) สอนจริยธรรม ALARA ต่อท้าย ผลของ Decree กระทบแค่ Hope/แรงงานในรอบนั้น ไม่เปลี่ยนจบจบ (ทุกเส้นทางที่ชนะไปจบที่ True Ending เดียว — ดู S14)

ดาบ 2 คม (เชื่อมกับเตา): แรงงานที่ได้มา = หล่อเย็นเพิ่ม = Boost ได้โดยไม่ Meltdown → ดัน CORE ถึง 100% ทันพายุ แต่ประกาศทั้ง 2 ข้อ Hope ร่วงเร็วมาก เสี่ยง Hope = 0 → แพ้ → ย้ายทเรียน Q10

→ ผูกควิช: Q10

12. ควิช 10 ข้อ เนื้อหาเต็ม (Quizzes)

รูปแบบควิช: 1 คำถาม + 3 ตัวเลือก **A / B / C** (ข้อความภาษาไทยทั้งหมด) · **ตอบถูก +8 Knowledge** · **ตอบผิด +3** · **ตอบ บังคับ ห้ามไม่ได้** (ต้องเลือกก่อนจึงไปต่อ) · **ตอบเสร็จแล้ว** ไม่ว่าถูกหรือผิด จะแสดง **"คำอธิบายความรู้"** อันเดียวกันเสมอ — ไม่อธิบายรายชื่อที่ผิด · คู่มือตามหมวดที่ §17 ในเอกสารนี้ ✓ = ข้อที่ถูก (ในเกมจริงลำดับตัวเลือกสุ่มผ่าน correctIndex)

ทุกข้อนำไปทำเป็น QuizQuestionSO (ดู §16) **ฟิลด์:** id, category, question, options[], correctIndex, rewardKnowledge, explainText, speaker

Quiz Master Map (ภาพรวมควิช 10 ข้อ)

ข้อ	ความรู้ที่สอน	เกิดเมื่อ
Q1	เชื้อเพลิงฟิวชัน — Deuterium	สัปดาห์ Deuterium ครั้งแรก · Day 11
Q2	พลาสมา & โทคาแมก	แก๊ววิกฤต 1 เสร็จ · ~Day 17
Q3	สนามแม่เหล็กคู่หล่อเย็น	กดใช้ Toroidal/Poloidal · ~Day 17
Q4	เวชศาสตร์นิวเคลียร์	แก๊ววิกฤต 2 เสร็จ · ~Day 20
Q5	หลัก ALARA ป้องกันรังสี	ส่งคนเข้าโซน A ครั้งแรก · ~Day 20
Q6	ปรับปรุงพันธุ์พืชด้วยรังสี	แก๊ววิกฤต 3 (เลือก A) · ~Day 24
Q7	ฉายรังสีถนอมอาหาร	ใช้โคบอลต์-60 (เลือก B) · ~Day 24
Q8 ★	ฟิวชันคืออะไร	จุดเตาฟิวชันติด · Day 25+
Q9 ★	ทำไมฟิวชันถึงสะอาด	ต่อจาก Q8 · Day 25+
Q10	จริยธรรม (Decree)	เปิดประกาศฉุกเฉิน · Day 25–30

ผู้บรรยาย: VESTA (AI ผู้ช่วยเตา) หรือ Dr. Auren Vasek (นักวิทยาศาสตร์) สลับกันตามข้อ — ระบุไว้ในแต่ละการ์ดด้านล่าง

Q1 · เชื้อเพลิงแรก: ดิวเทอเรียม · ผู้ถาม: VESTA

ถาม: ทำไมเราจึงสกัดเชื้อเพลิงตัวแรกได้จาก "น้ำ"?

- **A.** เพราะน้ำเป็นสารกัมมันตรังสีที่ปลอดภัยที่สุด
- **B.** เพราะน้ำมีดิวเทอเรียมอยู่แล้ว ไม่ต้องสร้างใหม่ จึงมีเชื้อเพลิงฟิวชันป้อนเตาได้ต่อเนื่อง ✓
- **C.** เพราะน้ำช่วยดับไฟในเตาไม่ให้ร้อน

คำอธิบาย: ดิวเทอเรียม (2H) เป็นไอโซโทปของไฮโดรเจนที่มีป้อนอยู่ในน้ำทั่วไปอยู่แล้ว เราจึงแยกออกมาใช้เป็นเชื้อเพลิงฟิวชันได้เลยโดยไม่ต้องผลิตขึ้นใหม่ ✦ ปลดล็อก Codex "Deuterium"

Q2 · ทำไมพลาสมาถึงพัง · ผู้ถาม: Dr. Auren Vasek

ถาม: อะไรคือสาเหตุที่ทำให้เตาเสี่ยงหลอมละลาย (Meltdown) เมื่อพลาสมาไม่เสถียร?

- A. พลาสมาที่ร้อนหลายล้านองศาหลุดออกไปชนผนังเตา ถ่ายเทความร้อนเข้าตัวอาคาร ✓
- B. พลาสมาเย็นเกินไปจนเตาหยุดทำงาน
- C. มีน้ำมากเกินไปจนเตาจม

คำอธิบาย: ในโทคาแมก พลาสมาที่ร้อนหลายล้านองศาถูกกักไว้ด้วยสนามแม่เหล็ก ถ้าสนามไม่นิ่งจนพลาสมาหลุดไปชนผนัง จะถ่ายเทความร้อนเข้าตัวอาคารจนเสี่ยงหลอมละลาย ♦ ปลดล็อก Codex "Plasma Confinement"

Q3 · สนามแม่เหล็กคู่คืออะไร · ผู้ถาม: VESTA

ถาม: เมื่อ HEAT ของเตาเริ่มได้สูง ควรทำอะไรเพื่อกันไม่ให้ถึง Meltdown?

- A. ป้อนอาหารเพิ่มให้คนงาน
- B. ปิดโรงน้ำเพื่อประหยัดพลังงาน
- C. เสริมสนามแม่เหล็ก/หล่อเย็น (Toroidal-Poloidal) เพื่อยกเพดานและรีดความร้อนออก ✓

คำอธิบาย: สนามแม่เหล็กคู่ทำงานร่วมกัน — Toroidal บีบพลาสมาให้วิ่งเป็นวง ส่วน Poloidal กันไม่ให้พลาสมาชนผนัง เมื่อเสริมให้แข็งแรงจะกักพลาสมาไว้กลางเตาและลดความร้อนที่รั่วออก ♦ ปลดล็อก Codex "Magnetic Confinement"

Q4 · เวชศาสตร์นิวเคลียร์: หาก่อน แล้วค่อยยิง · ผู้ถาม: แพทย์ประจำเมือง

ถาม: เวชศาสตร์นิวเคลียร์จัดการเซลล์เนื้อร้ายเป็น 2 ขั้นตอน — ข้อใดอธิบายได้ถูกต้อง?

- A. PET/SPECT ยิงรังสีทำลายเนื้อร้ายทันทีตั้งแต่ตอนสแกน
- B. PET/SPECT ใช้สารรังสี "ถ่ายภาพ" หาตำแหน่งเซลล์ผิดปกติก่อน จากนั้นยาเฉพาะจุดจึงส่งรังสีไป "ทำลาย" เป้าได้แม่นยำ กระทบเนื้อดีน้อย ✓
- C. รังสีฆ่าทุกเซลล์ในร่างกายเท่ากันหมด ไม่ว่าจะดีหรือร้าย

คำอธิบาย: เวชศาสตร์นิวเคลียร์ทำงาน 2 ขั้นตอน — (1) วินิจฉัย: PET/SPECT ฉีดสารเภสัชรังสีเพื่อ "ถ่ายภาพ" หาตำแหน่งเซลล์ผิดปกติ (ยังไม่ได้รักษา) (2) รักษา: ยาเฉพาะจุด (targeted therapy) จึงส่งรังสีไปทำลายเฉพาะเป้า กระทบเนื้อดีรอบ ๆ น้อย — ความแม่นยำคือหัวใจของทั้งสองขั้น ♦ ปลดล็อก Codex "Nuclear Medicine"

Q5 · ใครห้ามเข้าเขตรังสี (ALARA) · ผู้ถาม: VESTA

ถาม: ตามหลัก ALARA เมื่อต้องส่งคนเข้าพื้นที่เสี่ยงรังสี ควรจัดการอย่างไร?

- A. จำกัดเวลา/ปริมาณรังสีต่อคนให้น้อยที่สุด และเลี้ยงส่งกลุ่มเปราะบาง (ผู้ป่วย/เด็ก) ✓
- B. ส่งใครก็ได้เข้าไปนานเท่าไรก็ได้ ถ้ามีชุดกันรังสี
- C. ส่งผู้ป่วยเข้าไปก่อน เพราะป่วยอยู่แล้ว

คำอธิบาย: ALARA (As Low As Reasonably Achievable) คือ "ให้คนรับรังสีน้อยที่สุดเท่าที่ทำได้" และต้องปกป้องกลุ่มที่ไวต่อรังสีเป็นพิเศษ (ผู้ป่วย/เด็ก) ก่อนเสมอ ♦ ปลดล็อก Codex "ALARA"

Q6 · แก๊สที่ต้นเหตุ ไม่ใช่ปลายเหตุ · ผู้ถาม: Dr. Auren Vasek

ถาม: การฉายรังสีใส่เมล็ดพันธุ์ช่วยแก้วิกฤตอาหารระยะยาวได้อย่างไร?

- A. ทำให้พืชเรืองแสงจึงปลูกตอนกลางคืนได้
- B. ทำให้พืชกลายเป็นกัมมันตรังสี กินแล้วแข็งแรง
- C. กระตุ้นการกลายพันธุ์เชิงบวก คัดสายพันธุ์ที่ทนทาน/ผลผลิตสูงไว้ใช้ถาวร ✓

คำอธิบาย: การฉายรังสีกระตุ้นให้เกิดการกลายพันธุ์ นักวิจัยคัดเลือกเฉพาะสายพันธุ์ที่ทนทานและให้ผลผลิตสูงไว้ใช้ถาวร เช่น ข้าว กข6 ของไทย — เป็นการแก้ปัญหาที่ต้นเหตุ ♦ ปลดล็อก Codex "Mutation Breeding"

Q7 · หยุตอาหารเน่าด้วยรังสี · ผู้ถาม: VESTA

ถาม: การฉายรังสีแกมมาจากโคบอลต์-60 ช่วยถนอมอาหารด้วยกลไกใด?

- A. รังสีเคลือบผิวอาหารด้วยโลหะกันบูด
- B. รังสีทำลายจุลินทรีย์/เชื้อราในอาหาร ชะลอการเน่า โดยอาหารไม่กลายเป็นสารรังสี ✓
- C. รังสีทำให้อาหารแช่แข็งตลอดเวลา

คำอธิบาย: รังสีแกมมาทะลุผ่านอาหารและฆ่าจุลินทรีย์/เชื้อรา ทำให้เก็บได้นานขึ้น โดยอาหารไม่กลายเป็นสารกัมมันตรังสี (อาหารฉายรังสี ≠ อาหารมีรังสี) ♦ ปลดล็อก Codex "Food Irradiation"

Q8 ★ · ฟิวชันคืออะไร · ผู้ถาม: VESTA

ถาม: "ฟิวชันนิวเคลียร์" ที่เพิ่งจุดติดในเตา — แท้จริงคืออะไร?

- A. การหลอมรวมนิวเคลียสเบาเข้าด้วยกัน (ไฮโดรเจน + ไฮโดรเจน) ให้เป็นธาตุที่หนักกว่า แล้วปล่อยพลังงานมหาศาล ✓
- B. การแตกตัวของนิวเคลียสหนักออกเป็นชิ้นเล็ก เหมือนเครื่องปฏิกรณ์ฟิชชัน
- C. การเผาไหม้ถ่านหินด้วยความร้อนสูง

คำอธิบาย: ฟิวชันคือการหลอมรวมนิวเคลียสเบา (เช่น ไฮโดรเจน) ให้กลายเป็นธาตุที่หนักกว่า แล้วปลดปล่อยพลังงานมหาศาล — ตรงข้ามกับฟิชชันที่เป็นการ "แตกตัว" ของนิวเคลียสหนัก ♦ ปลดล็อก Codex "Nuclear Fusion"

Q9 ★ · ทำไมฟิวชันถึงสะอาด · ผู้ถาม: Dr. Auren Vasek

ถาม: ข้อใดอธิบายได้ถูกต้องว่าทำไมพลังงานฟิวชันถึงสะอาดกว่าทางเลือกอื่น?

- A. เพราะมันไม่เกี่ยวข้องกับรังสีหรือนิวเคลียร์เลย บริสุทธิ์ 100%
- B. เพราะมันเผาถ่านหินที่สะอาดเป็นพิเศษ
- C. เชื้อเพลิงมาจากน้ำทะเล ไม่ปล่อย CO₂ ไม่มีปฏิกิริยาลูกโซ่ที่คุมไม่ได้ และไม่มีกากรังสีอายุยืนแบบฟิชชัน ✓

คำอธิบาย: ฟิวชันสะอาดกว่าเพราะเชื้อเพลิงหาได้จากน้ำ ไม่ปล่อย CO₂ ถ้าเสียสมดุลเตาจะดับเอง (ไม่ระเบิด) และไม่มีกากรังสีอายุยืนแบบฟิชชัน — แต่ "สะอาดกว่า" ไม่ได้แปลว่า "ไม่มีรังสีเลย" เพราะเชื้อเพลิง D-T ยังปล่อยนิวตรอน ♦ ปลดล็อก Codex "Clean Energy"

Q10 · ราคาที่ต้องไม่เกินขีด · ผู้ถาม: Dr. Auren Vasek

ถาม: การเกณฑ์แรงงานฉุกเฉินให้คนเข้าทำงานในเขตเสี่ยงรังสีขณะวิกฤต — กับคักทางจริยธรรมอยู่ตรงไหน?

- A. ไม่มีปัญหาอะไร เพราะคนกลุ่มนี้แค่ทำงานช้ากว่าคนทั่วไป
- B. กลุ่มเปราะบางไวต่อรังสีเป็นพิเศษ การบังคับให้เข้าไปเสี่ยงจึงขัดหลัก ALARA โดยตรง ✓
- C. ไม่ขัดอะไรเลย ถ้าทุกคนในเมืองตกลงร่วมมือกัน

คำอธิบาย: การส่งผู้ป่วย/เด็กเข้าเขตรังสีขัดหลัก ALARA เพราะคนกลุ่มนี้ไวต่อรังสีเป็นพิเศษ การตัดสินใจนี้เป็นของผู้เล่น เกมไม่ตัดสินถูก-ผิดแทน แต่ผลของมันคือ Hope และแรงงานในรอบนั้น ♦ เชื่อมโยงกับ ALARA (Q5)

13. ไอเทม & คราฟต์ (Items)

ของที่คราฟต์/ผลิตเพื่อดันค่า Q หรือพลิกวิกฤต (ผลิตที่ห้องวิจัย/โรงพยาบาล)

ไอเทม	ประเภท	ผลิตจาก	ใช้ทำอะไร
Deuterium	เชื้อเพลิง	สกัดจากน้ำ (โรงน้ำ L3)	จุดเตา + ดัน Q (เฟส 2-3 เตา)
Tritium	เชื้อเพลิง	Zone B (Phase 4)	ดัน Q ท้ายเตา (เชื้อเพลิงชั้นสูง)
Rad-Gear	อุปกรณ์	Iron + วิจัย	กันคนป่วย/ตายเมื่อส่งเข้าพื้นที่รังสี
เครื่องสแกน PET/SPECT	การแพทย์	โรงพยาบาล + พลังงาน	ถ่ายภาพหาตำแหน่งเซลล์ผิดปกติ — วินิจฉัย (วิกฤต 2 · A)
ไอโซโทปการแพทย์ (ยาเฉพาะจุด)	การแพทย์	วัสดุเล็บ + ฟลักซ์นิวตรอนจากเตา (เตา Idle 1 วัน)	รักษาผู้ป่วยหายสนิท (วิกฤต 2 · B)
เมล็ดพันธุ์ฉายรังสี	เกษตร	ห้องวิจัย + แร่เหล็ก 250	เพิ่มผลผลิตอาหารยาว (วิกฤต 3 · A)
เครื่องฉายโคบอลต์-60	ถนอมอาหาร	โรงงาน + พลังงาน 300	ยืดอาหารเน่า → 0% (วิกฤต 3 · B)
น้ำหล่อเย็นฉุกเฉิน	ฉุกเฉิน	น้ำสะอาด	ลด HEAT เตารั้งด่วน (วิกฤต 1 · C)
★ โล่พลาสมา	ชั้นชนะ	CORE TOWER ที่ $Q \geq 1.0$	กางรับพายุรังสี = ชนะเกม

หมายเหตุฟิสิกส์: ฟิวชัน D-T ปลอ่ย นิวตรอน ออกมา — และ "ฟลักซ์นิวตรอน" จากเตานี้แหละที่ใช้ผลิต ไอโซโทปการแพทย์จริง (เช่น Mo-99/Tc-99m, Lu-177, I-131 จากการอาบนิวตรอน) จึงเป็นเหตุผลที่วิกฤต 2 · B ใช้เวลาเดินเตา (Idle 1 วัน) มาผลิตยา — ตอกย้ำด้วยว่า "สะอาดกว่า" ไม่ใช่ "ไม่มีรังสีเลย" (เชื่อม Q9)

14. เฟส & ฉากจบ (Phases & Endings)

4 เฟสปลดล็อก

ช่วง	เฟส	ปลดล็อก
Day 1-5	PHASE 1 · เอาตัวรอด	โรงไฟ · โรงน้ำ · โรงอาหาร · เหมือง — ทำให้คลังไม่ติดลบ
Day 6-10	PHASE 2 · พัฒนา	ฝึกวิศวกร · ห้องวิจัย · สกัด Deuterium — เริ่มเตรียมเชื้อเพลิง
Day 11-20	PHASE 3 · นิวเคลียร์	Zone A · โรงพยาบาล · CORE TOWER — จุดเตา + เจอวิกฤตหนัก
Day 21-30	PHASE 4 · จุดติดเตา	Zone B · Tritium · โล่พลาสมา — ดัน Q ให้ถึง 1.0 ทันพายุ

Victory Loop (จังหวะมาตรฐาน 1 รอบเกม)

วัน	เหตุการณ์	ควิช
Day 11	จุดเตา · เริ่มสกัด Deuterium	Q1
Day 17	⚠ วิกฤต 1 (สนามแม่เหล็กคู่) → เข้าเฟส 2 เตา	Q2, Q3
Day 20	⚠ วิกฤต 2 (โรคกลายพันธุ์ Zone A)	Q4, Q5
Day 24	⚠ วิกฤต 3 (เสบียงเน่า)	Q6, Q7
Day 25	พายุเริ่ม · เปิด Tritium · Decree โฟล์	Q10
Day 25-30	FIRST LIGHT — ดัน 80→100% · จุดติด	Q8, Q9

ฉากจบ 3 แบบ

มี 3 ฉากจบ ทุกเส้นทางที่ชนะไปจบที่ True Ending เดียว (การใช้ Decree กระทบแค่ Hope/แรงงานในรอบนั้น บันทึกเป็นสถิติได้แต่ไม่เปลี่ยนบทจบ)

เงื่อนไข	ฉากจบ
Hope ถึง 0 ก่อน Day 30 / เตา Meltdown	Game Over — เมืองล่มสลายระหว่างทาง
Day 30 · Q 0.5-0.99	Normal Ending — โล่กางได้บางส่วน เมืองบาดเจ็บ
Day 30 · Q ≥ 1.0 + คนรอด	* True Ending — จุดเตาฟิวชันสำเร็จ กางโล่พลาสมารับพายุ

★ ระบบสรุปสาเหตุแพ้ (Defeat Summary) — แบบคร่าว ๆ

เมื่อแพ้ ให้ขึ้นจอสรุปสั้น ๆ ว่า "ทำไมรอบนี้ถึงแพ้" + สถิติย่อ ไม่ต้องทำซับซ้อน — แค่อธิบายเหตุที่ทำให้จบเกมแล้วแสดงข้อความตรงกับสาเหตุนั้น:

สาเหตุ (lossReason)	ข้อความสรุป
HopeZero	"ขวัญเมืองหมด (Hope = 0) — ประชาชนหมดศรัทธา เมืองล่มสลาย"
Meltdown	"เตาหลอมละลาย (HEAT ≥ 100) — แรงเครื่องเกินกำลังหล่อเย็น"
TimeoutLowQ	"หมดเวลา 30 วัน แต่ค่า Q = {Q} ยังไม่ถึง 1.0 — โล่พลาสมาไม่สำเร็จ"

แสดงเพิ่มแบบสั้น: วันที่ไปถึง · Q สุดท้าย · Knowledge ที่ได้อ่าน · ปุ่ม "เริ่มใหม่"

★ การเริ่มเกมใหม่ (Restart / Reset)

กด "เริ่มใหม่" → รีเซ็ตทรัพยากรทุกอย่างกลับค่าเริ่มต้น (Energy/Water/Food/Iron/เชื้อเพลิง/Hope/อาคาร/ประชากร/CORE%/HEAT/วัน) ยกเว้น "คลังความรู้" (Knowledge bank + Codex ที่ปลดล็อกแล้ว) ที่คงอยู่ถาวร (ดู §9) — ผู้เล่นจึงเก่งขึ้นเรื่อย ๆ จากความเข้าใจที่สะสม

15. ★ ระบบใหม่: หยุดเวลาตอนวางอาคาร (Placement Pause System)

เป้าหมาย (จากที่ร้องขอ): เมื่อผู้เล่นกดเลือกอาคารจากเมนูสร้าง (เช่น ลาก "ห้องวิจัย" มาวางในเกม) เวลาในเกมต้องหยุด ให้ผู้เล่นได้คิด/วิเคราะห์ว่าจะวางตรงไหน จนกว่าจะวางเสร็จ (หรือยกเลิก) เวลาถึงจะเดินต่อ

พฤติกรรมที่ต้องได้ (Behavior Spec)

- ผู้เล่นกดอาคารใน **Build Menu** (หรือเริ่มลาก) → เข้าสู่ **Placement Mode**
- ทันทีที่เข้า Placement Mode: - นาฬิกา**วันหยุด** (ไม่ว่าจะกำลังอยู่เฟส Planning 30s หรือ Live 60s) · เหตุการณ์/**วิกฤตไม่แจ้ง** - แสดง **ghost preview** ของอาคารเกาะตามเคอร์เซอร์ - แสดงต้นทุน + สถานะวางได้/ไม่ได้ (เขียว/แดง)
- ผู้เล่นเลื่อน ghost ไปมา — เวลา**ยังหยุด** ไม่มีทรัพยากรไหล อ่าน tooltip/วิเคราะห์ผังได้เต็มที่
- ยืนยันวาง** (คลิกช่องที่วางได้) → วางอาคาร, หักต้นทุน, **เวลาเดินต่อ หรือ ยกเลิก** (คลิกขวา / Esc / ปุ่ม X) → ghost หาย, ไม่หักต้นทุน, **เวลาเดินต่อ**
- วางได้ที่ละ 1 อาคาร · ควิซ/วิกฤต popup ก็หยุดเวลาด้วย (สอดคล้องดีไซน์ "ไม่กดดันเวลาตอนคิด")

เงื่อนไขวางได้ (Placement Validity)

ghost จะเป็น **สีเขียว (วางได้)** ก็ต่อเมื่อครบทุกข้อ มิฉะนั้น **สีแดง (วางไม่ได้)** และกดยืนยันไม่ได้:

- ช่องว่าง ไม่ทับอาคารอื่น/สิ่งกีดขวาง
- ทรัพยากรพอจ่ายต้นทุนสร้าง
- เฟส/โซนปลดล็อกแล้ว (เช่น Zone B ต้อง Phase 4)
- (ถ้ามี) อยู่ในพื้นที่ที่อนุญาตของอาคารชนิดนั้น

State Machine

กดอาคารใน Build Menu

```
+-----+ +-----+
| Idle |           | Placing |
|เวลาเดิน | <----- |เวลาหยุด |
+-----+   วาง(valid) / ยกเลิก   +-----+

                                |   เลื่อน ghost = อัปเดตสีเขียว/แดง
                                +---- (ยังอยู่ Placing, เวลายังหยุด)
```

Pseudo-flow (C#)

```
// เข้าโหมดวาง: เรียกเมื่อผู้เล่นเลือกอาคารจาก Build Menu
void BeginPlacement(BuildingSO building) {
    if (state == PlacementState.Placing) CancelPlacement(); // place one at a time
    pendingBuilding = building;
    ghost = SpawnGhost(building);
    TimeManager.Instance.Pause(PauseReason.Placement); // <-- freeze game time
    state = PlacementState.Placing;
}

// called every frame while Placing
void UpdatePlacement(Vector3 cursorWorldPos) {
    Vector2Int tile = GridUtil.WorldToTile(cursorWorldPos);
    ghost.MoveTo(tile);
    bool valid = IsTileFree(tile)
        && ResourceManager.Instance.CanAfford(pendingBuilding.buildCost)
        && PhaseManager.Instance.IsUnlocked(pendingBuilding);
    ghost.SetTint(valid ? Color.green : Color.red);
    lastValid = valid;
}

// confirm placement (left-click on a valid tile)
void ConfirmPlacement(Vector2Int tile) {
    if (!lastValid) return; // block invalid placement
    ResourceManager.Instance.Spend(pendingBuilding.buildCost);
    BuildingManager.Instance.PlaceBuilding(pendingBuilding, tile);
    EndPlacement();
}

// cancel (right-click / Esc / X button)
void CancelPlacement() { EndPlacement(); }

void EndPlacement() {
    Destroy(ghost);
    pendingBuilding = null;
    state = PlacementState.Idle;
    TimeManager.Instance.Resume(PauseReason.Placement); // <-- resume game time
}
```

หมายเหตุการ implement

- ใช้ระบบ **pause-reason** (ดู `TimeManager §16`) ห้ามตั้ง **`Time.timeScale = 0`** ตรง ๆ เพราะจะหยุดทั้ง UI/animation — ให้หยุดเฉพาะ "นาฬิกาวัน" ผ่าน flag `IsRunning` แทน เพื่อให้ ghost ยังเคลื่อนได้ลื่นและ UI ยังตอบสนอง
- pause-reason ต้องซ้อนกันได้ (Placing + Quiz popup พร้อมกัน) → เวลาจะเดินต่อเมื่อ **ไม่เหลือ reason ใดเลย**

- ใช้ได้ทั้งเฟส Planning (30s) และ Live (60s) — เปิด Build Menu เมื่อไรนาฟีกาก็หยุดทันที ผู้เล่นวางอาคารได้สบาย ๆ โดยไม่เสียวินาทีของวัน

16. สถาปัตยกรรม Unity + โครงสร้างคลาส C# (Architecture)

Manager / Service (singleton หรือ service locator)

คลาส	หน้าที่
GameManager	สแตตแมชชีนภาพรวม (เฟส, นับวัน, เช็ค win/lose, สาเหตุแพ้ + จอสรุป)
TimeManager	นาฬิกาวัน 90s (Planning 30s + Live 60s), pause-reason stack
ResourceManager	คลังทรัพยากรทุกชนิด + Hope + Knowledge + ผลิต/บริโภครายวัน
PopulationManager	Worker/Engineer/Medic, การจัดสรร, เพดาน Shelter
BuildingManager	ทะเบียนอาคารที่วางบน กริด 43x43 + คำนวณผลผลิตแบบจบวันที่เดียว (batch)
BuildPlacementManager	ระบบ ghost/วาง/หยุดเวลา (\$15)
ReactorController	CORE%, HEAT, mode (ล็อกตอน Planning), เชื้อเพลิง, สูตรหล่อเย็น (\$8)
CrisisManager	trigger, resolve, ผูกควิซผ่าน IQuizTrigger
QuizManager	แสดง popup ควิซ (ตอบบังคับ ข้ามไม่ได้), เพิ่ม Knowledge, กันถามซ้ำ
DecreeManager	ประกาศฉุกเฉิน (\$11)
PhaseManager	จัดการเฟส 1-4 + เงื่อนไขปลดล็อกอาคาร
MetaProgress	คลังความรู้ถาวร — เก็บ Knowledge + Codex ข้ามรอบเล่น (รอด restart)
UIManager	HUD, popup, จอสรุปสาเหตุแพ้ (Defeat Summary)
AudioManager	เสียง (ออปชั่น)

Data — ScriptableObject (โครง C# พร้อมต่อยอด)

```
// ===== ResourceCost / ResourceAmount (helper structs) =====
public enum ResourceType { Energy, Water, Food, Iron, Deuterium, Tritium }

[System.Serializable]
public struct ResourceAmount { public ResourceType type; public int amount; }

[System.Serializable]
public class ResourceCost { public ResourceAmount[] items; } // e.g. Energy 150 + Iron 40

// ===== BuildingSO =====
public enum BuildingCategory { Production, Research, Reactor, Support }

[CreateAssetMenu(menuName = "NuclearReMind/Building")]
public class BuildingSO : ScriptableObject {
    public string id;
    public string displayName;
    public BuildingCategory category;
    public int workersRequired;
    public ResourceCost buildCost;           // one-time cost to construct
    public ResourceCost upkeepPerDay;        // daily running cost
    public ResourceAmount[] productionPerDay; // outputs per day at current level
    public Phase unlockPhase;                // when it becomes buildable
    public Vector2Int footprint;              // tile size for placement
    public BuildingLevelSO[] levels;         // L1..L3 stats (optional per type)
}

// ===== CrisisSO =====
[CreateAssetMenu(menuName = "NuclearReMind/Crisis")]
public class CrisisSO : ScriptableObject, IQuizTrigger {
    public string id;
    public string title;
    [TextArea] public string situationText;
    public TriggerCondition trigger;          // e.g. Heat>80, Day>=N, FoodStored>500
    public int daysToResolve;
    public CrisisChoice[] choices;           // A / B / C
    public QuizQuestionSO[] linkedQuizzes;

    public QuizQuestionSO[] GetLinkedQuizzes() => linkedQuizzes;
}

[System.Serializable]
public class CrisisChoice {
    public string label;                     // "A · เร่งสนามแม่เหล็กวงแหวน"
    public ResourceCost cost;
    public CrisisEffect[] effects;           // resource/hope/population deltas
}
```

```

    [TextArea] public string outcomeText;    // result shown to player
}

// ===== QuizQuestionSO =====
public enum QuizCategory { Reactor, Agriculture, Medical, Ethics }

[CreateAssetMenu(menuName = "NuclearReMind/QuizQuestion")]
public class QuizQuestionSO : ScriptableObject {
    public string id;                        // "Q1".. "Q10"
    public QuizCategory category;           // drives icon/colour (see §17)
    [TextArea] public string question;
    public string[] options;                // 3 choices
    public int correctIndex;                // index of the correct option
    public int rewardKnowledge = 8;         // +8 correct, see explainText for wrong (+3)
    [TextArea] public string explainText;   // shown for BOTH correct & wrong
    public string speaker;                  // "VESTA" / "Dr. Auren Vasek" / ...
}

// ===== DecreeSO =====
[CreateAssetMenu(menuName = "NuclearReMind/Decree")]
public class DecreeSO : ScriptableObject, IQuizTrigger {
    public string id;
    public string title;
    public int coolingLaborGain;             // +cooling workers granted
    public int hopeImmediate;               // negative
    public int hopePerDay;                  // negative, while active
    public float dailyDeathChance;          // e.g. 0.20 for sick-labor decree
    public QuizQuestionSO[] linkedQuizzes; // Q10
    public QuizQuestionSO[] GetLinkedQuizzes() => linkedQuizzes;
}

```

Interface กลางสำหรับฝั่งควิช

```

public interface IQuizTrigger {
    QuizQuestionSO[] GetLinkedQuizzes();
}

// CrisisSO -> Q2,Q3 / Q4,Q5 / Q6,Q7
// TechUnlock-> Q1    · IgnitionEvent -> Q8,Q9    · DecreeSO -> Q10

```

TimeManager (หัวใจของระบบหยุดเวลา §3, §15)


```

public enum PauseReason { Placement, QuizPopup, CrisisPopup, MainMenu, Manual }
public enum DayPhase { Planning, Live }

public class TimeManager : MonoBehaviour {
    public static TimeManager Instance;

    public int CurrentDay { get; private set; } = 1;
    public DayPhase Phase { get; private set; } = DayPhase.Planning;

    // 1 day = 90s total (30s planning + 60s live)
    [SerializeField] private float planningPhaseSeconds = 30f;
    [SerializeField] private float livePhaseSeconds      = 60f;
    private float phaseTimer;
    private readonly HashSet<PauseReason> activePauses = new();

    // Day clock advances ONLY when no pause reason is active (placement / any popup)
    public bool IsRunning => activePauses.Count == 0;

    public void Pause(PauseReason r) => activePauses.Add(r);
    public void Resume(PauseReason r) => activePauses.Remove(r);

    void Start() => BeginPlanning();

    void BeginPlanning() { Phase = DayPhase.Planning; phaseTimer = planningPhaseSeconds; }
    void BeginLive() {
        Phase = DayPhase.Live; phaseTimer = livePhaseSeconds;
        ReactorController.Instance.LockMode(); // mode chosen in planning is locked for the day
    }

    void Update() {
        if (!IsRunning) return; // frozen during placement / popups
        phaseTimer -= Time.deltaTime;
        if (phaseTimer > 0f) return;
        if (Phase == DayPhase.Planning) BeginLive(); // 30s planning done -> 60s live
        else EndDay(); // 60s live done -> resolve the day
    }

    void EndDay() {
        // Production is computed ONCE per day (batch) – not accrued frame-by-frame
        BuildingManager.Instance.ApplyDailyProduction(); // outputs - upkeep (batch)
        ResourceManager.Instance.ApplyDailyConsumption(); // food/water per person
        ReactorController.Instance.ApplyDailyTick(); // ΔCORE% / ΔHEAT ($8)
        CrisisManager.Instance.CheckTriggers(); // crisis + embedded quiz (pauses
time)
        GameManager.Instance.CheckWinLose();
    }
}

```

```
        CurrentDay++;  
        BeginPlanning(); // next day  
    }  
}
```

ResolveCrisis flow (ฟังวิชต่อท้ายวิกฤต)

```
void ResolveCrisis(CrisisSO crisis, int choiceIndex) {  
    ApplyChoiceEffects(crisis.choices[choiceIndex]); // result of A/B/C  
    foreach (var quiz in crisis.GetLinkedQuizzes()) {  
        if (!QuizManager.Instance.AlreadyAnswered(quiz.id))  
            QuizManager.Instance.ShowQuizPopup(quiz); // mandatory popup: no skip button  
    }  
    EndCrisis();  
}
```

Persistence & Defeat Summary (คลังความรู้ถาวร + จอสรุปแพ้)

```
// ===== MetaProgress: persists ACROSS runs (PlayerPrefs or a meta save file) =====
// Everything else resets on restart; this does NOT.
public static class MetaProgress {
    public static int KnowledgeBank; // accumulated Knowledge kept
    public static HashSet<string> UnlockedCodex = new(); // codex ids already read
    public static void Save() { /* write to PlayerPrefs / file */ }
    public static void Load() { /* read on game launch */ }
}

// ===== GameManager: win/lose + defeat summary =====
public enum LossReason { None, HopeZero, Meltdown, TimeoutLowQ }

public void CheckWinLose() {
    if (ResourceManager.Instance.Hope <= 0) EndGame(LossReason.HopeZero);
    else if (ReactorController.Instance.Heat >= 100) EndGame(LossReason.Meltdown);
    else if (TimeManager.Instance.CurrentDay > 30) {
        float q = ReactorController.Instance.Q;
        if (q >= 1.0f) ShowEnding(Ending.True);
        else if (q >= 0.5f) ShowEnding(Ending.Normal);
        else EndGame(LossReason.TimeoutLowQ);
    }
}

void EndGame(LossReason reason) {
    // keep Knowledge for next run; all other state will be reset on Restart
    MetaProgress.KnowledgeBank =
        Mathf.Max(MetaProgress.KnowledgeBank, ResourceManager.Instance.Knowledge);
    MetaProgress.Save();
    UIManager.Instance.ShowDefeatSummary(reason); // short "why you lost" screen (§14)
}

// Restart(): reload scene / reset every manager to start values (§4, §5),
// then ResourceManager.Knowledge = MetaProgress.KnowledgeBank; // only knowledge carries over
```

ประเมินงานเพิ่ม (Effort)

- **ระบบควิช:** ~1 ScriptableObject + IQuizTrigger + ฟิลต์ในเหตุการณ์เดิม + Popup เดิม + 1 ฟิลต์ Knowledge ใน ResourceManager ~ งานโปรแกรมเมอร์ 1–2 วัน ไม่กระทบระบบอื่น
- **ระบบหยุดเวลาตอนวางอาคาร (§15):** BuildPlacementManager + เพิ่ม pause-reason ใน TimeManager ที่มีอยู่ ~ 0.5–1 วัน (ถ้า TimeManager ออกแบบด้วย flag IsRunning ตั้งแต่ต้น)

17. UI / UX Spec

หลัก UX สำคัญ

- ใช้ **Popup เดิมร่วมกัน**: Decision Quiz ใช้ Popup ตัวเดียวกับระบบเลือก A/B/C ที่มีอยู่ (UIManager/HUD Popup) แต่เพิ่มเนื้อในข้างใน ไม่ต้องเขียนจอใหม่
- **ไม่มีปุ่มข้าม — คิวชตอบบังคับ** ผู้เล่นต้องเลือกคำตอบก่อนถึงปิด popup ได้ (ปุ่มยืนยันใช้งานได้ต่อเมื่อเลือกตัวเลือกแล้ว)
- **ตอบเสร็จแล้วแสดง "คำอธิบายความรู้" อันเดียวกันเสมอ ไม่ว่าจะตอบถูกหรือผิด** — ไฮไลต์ข้อที่ถูก (เขียว) + ข้อที่ผู้เล่นเลือกผิด (แดง) แล้วโชว์คำอธิบายความรู้ที่ถูกต้อง 1 ย่อหน้า (ไม่อธิบายแยกรายข้อผิด) เน้นให้เข้าใจง่าย
- **ไม่มีตัวจับเวลาแบบกดคัต** ในคิวช/วางอาคาร — ให้ผู้เล่นคิด ไม่ไชรีบ (เชื่อมระบบ pause S15)

สีตามหมวดคิวช (QuizCategory)

หมวด	สี	ใช้กับ
Reactor (เตา/ฟิวชั่น)	น้ำเงิน	Q1, Q2, Q3, Q8, Q9
Agriculture (เกษตร)	เขียว	Q6, Q7
Medical (แพทย์/รังสี)	แดง	Q4
Ethics (จริยธรรม)	เทา	Q5, Q10

HUD ควรแสดงตลอด

แถบทรัพยากร 6 ชนิด · Hope · Knowledge · วันที่ปัจจุบัน + เฟส + **ตัวนับเวลาวัน (90s)** · **CORE% + HEAT (แถบเตา)** · ปุ่มเลือกโหมดเตา (Idle/Normal/Boost/Overdrive ล็อกตอน Planning) + ปุ่ม SCRAM · ปุ่ม Build Menu · ปุ่ม "เริ่มวัน" (กดข้ามเวลาวางแผนที่เหลือเข้าสู่ Live ได้เลย)

จออื่น ๆ

- **จอสรุปสาเหตุแพ้ (Defeat Summary)**: ข้อความสั้นตามสาเหตุ (HopeZero/Meltdown/TimeoutLowQ) + สถิติย่อ + ปุ่ม "เริ่มใหม่" (ดู S14)
- **จอบจบเกมชนะ (Ending)**: Normal / True Ending (ดู S14)

18. ★ ตารางค่าปรับสมดุลทั้งหมด (Tuning Constants)

รวมทุก "ตัวเลขเวท" ไว้ที่เดียว แนะนำทำเป็น `GameConfigSO` ScriptableObject ตัวเดียวเพื่อจูนได้โดยไม่แตะโค้ด

ค่าเริ่มต้น & ประชากร

คีย์	ค่า
startEnergy / startWater / startFood / startIron	200 / 150 / 150 / 100
startHope	100 (max 100, lose at 0)
startPopulation	10 (all Workers)
foodCapBeforeSpoilage	500
consumeFoodPerPerson / consumeWaterPerPerson	2 / 2 ต่อวัน
populationGrowthPerDay	+1 (ถ้า Food เหลือ & Hope $\geq$ 50)
trainEngineerCost / trainMedicCost	Food 30+Energy 50 / Food 40+Energy 60 (1 วัน)

Shelter

ระดับ	เพดาน	ต้นทุน
L1 / L2 / L3 / L4	10 / 20 / 40 / 80	-, Iron60+E100, Iron120+E250, Iron200+E450

Production (ผลิต/วัน · L1/L2/L3)

อาคาร	L1	L2	L3
Power	+60	+180	+450
Water	+50	+150	+375
Food	+40	+120	+300
Mine	+30	+90	+225
Agri Dome	+150 (flat)	—	—

Upkeep (ค่าเดินระบบ/วัน)

อาคาร	upkeep
Power	0
Water L1/L2/L3	E10 / E25 / E50
Food L1/L2/L3	E8+W15 / E16+W30 / E30+W60
Mine L1/L2/L3	E12 / E20 / E36
Research / Hospital / Zone B	E20 / E15 / E30+W30
Agri Dome	E40 + W40

Reactor (เตา)

คีย์	ค่า
coreStartPercent	30 (Day 11)
coreWinPercent	100 (= Q 1.0)
baseCorePerTick	3
modeMultiplier Idle/Normal/Boost/Overdrive	0 / 1 / 2 / 3
heatFromMode Idle/Normal/Boost/Overdrive	0 / 5 / 20 / 40
fuelNeed Normal/Boost/Overdrive	10 / 20 / 30 ต่อวัน
baseCooling	15
coolingPerEngineer	4 (ต้องมี Poloidal Coils)
coolingTowerPerLevel	10 (cap 3 ระดับ = +30) — Toroidal Coils
heatFromStorm (Day 25–30)	+12 / วัน
heatWarnZone / heatMeltdown	80 / 100
microDamagePerTick (HEAT 80–99, ไม่มี Poloidal)	CORE% –2
knowBonus (Knowledge ≥ 80)	+0.10
SCRAM	HEAT –40, CORE% –10, Water –50, cooldown 2 วัน

Hope deltas

เหตุการณ์	ค่า
อาหารขาด	–10/วัน
คนตาย	–5/คน
แก้วิกฤตสำเร็จ	+5
Medic + ไม่ขาดของ	+2/วัน
Decree 1	–8 ทันที, –3/วัน · ตาย 20%/วัน
Decree 2	–15 ทันที

Knowledge

การกระทำ	ค่า
ตอบถูก / ตอบผิด	+8 / +3
อ่าน Codex	+2
ผ่านวิกฤต	+5
Thresholds	0–29 Novice · 30–59 Aware(วิจัย –10% E) · 60–79 Skilled(eng เร็วขึ้น) · 80–100 Expert(knowBonus +0.10)

Crisis triggers (เวลาโดยประมาณ)

วิกฤต	trigger	~วัน
Plasma Instability	HEAT > 80 หรือ Q > 0.3	Day 17
Outbreak	ส่งคนชุด Zone A เกิน → 15 คนป่วย	Day 20
Food Crisis	Food > 500 หรือไม่มี Agri Dome	Day 24
Storm	เข้า Phase 4 / Ignition	Day 25–30

Time & Map

คีย์	ค่า
planningPhaseSeconds	30
livePhaseSeconds	60
1 วัน	90s (Planning 30 + Live 60)
production	คำนวณจบวันที่เดียว (batch ที่ EndDay)
reactor mode	ล็อกตอน Planning เดินตลอด Live
total days	30 (Day 1 = tutorial)
gridSize	43 × 43 ช่อง
coolingWaterPool	ใช้คูลิ่งน้ำเดียวกับน้ำดื่ม (ไม่มีถังแยก)

Persistence (คลังความรู้ถาวร)

คีย์	ค่า
MetaProgress.KnowledgeBank	Knowledge สะสม คงข้ามรอบ (รอด restart)
MetaProgress.UnlockedCodex	Codex ที่อ่านแล้ว คงข้ามรอบ
restart รีเซ็ต	ทุกอย่าง ยกเว้น 2 ค่าข้างบน

19. คำถามที่ค้างไว้ให้ตัดสินใจ (Open Decisions)

สิ่งที่ยังไม่กำหนด (ไม่บล็อกการเขียนโค้ด — ทำทีหลัง/ตอนทำอาร์ตได้):

- 1. **footprint อาคารใหญ่** — CORE TOWER / Zone กินกี่ช่องบนกริด 43x43 (โครง footprint ใน S16 รองรับแล้ว ใส่ค่าตอนทำ prefab)
- 2. **ภาพ/อาร์ต isometric** — ขนาด sprite, ทิศทาง, จำนวนเฟรม animation, อีมีสี (อยู่นอกขอบเขตเอกสารนี้)
- 3. **เสียง** — BGM/SFX แต่ละเหตุการณ์ (ออกแบบ)
- 4. **ค่าจริงต้อง playtest** — ตัวเลขใน S18 "สมดุลงบกระดาศ" ควรเล่นจริง 2–3 รอบแล้วจูน โดยเฉพาะความตึงพลังงาน Phase 4 และจำนวนวัน Boost ที่จำเป็น

หมายเหตุ: ประเด็นที่เคยค้าง (production จบวันที่เดียว, โหมดเตาสีออกตอน Planning, กริด 43x43, น้ำหล่อเย็นคลังเดียว, ระบบเซฟคลังความรู้) ถูกกำหนดเรียบร้อยแล้วในเอกสารนี้

จบเอกสาร — NUCLEAR Re:Mind · Developer Handoff Edition